ASSIGNMENT - 4

High Performance Computing Lab (HPCL)

NAME	Aryan Shirsath
ENROLLMENT	MITU22BTCS0170
SUBJECT	HPCL
ASSIGNMENT TITLE	ASSIGNMENT - 4

MPI Programming – Four Practical Programs

QUESTION 1

Write a simpleMPI programtodisplay a Hello World message from each process. The program should initializeMPI, obtaintherankof each process and the total number of processes, print the message, andfinally finalizeMPI.

Theory / Explanation

MPI (Message PassingInterface)isused to run a program using multiple processes. Each process has a unique rank, startingfrom0.MPI_Comm_rank() gives the rank of the current process, while MPI_Comm_size() givesthetotalnumber of processes. When the program is executed with four processes, the ranks are0,1,2and3.

In thisprogram, MPI_Init()startstheMPI environment and MPI_Finalize() ends it. Each process printsits own rank andthetotalnumber of processes.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Variables for process rank and total number of processes
    int rank; int size;

    //    Get    the    rank    of    the    current    process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

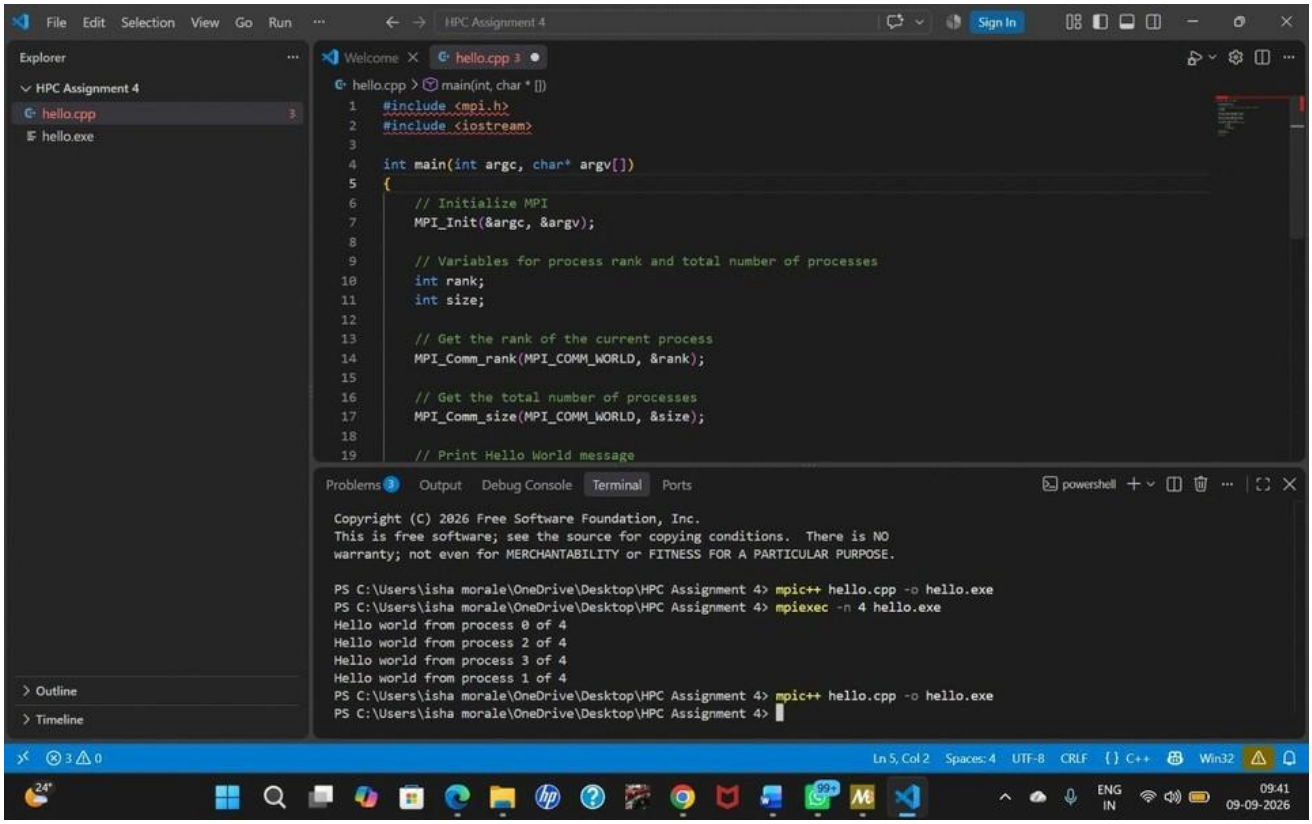
    //     Get     the     total     number     of     processes
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Print Hello World message std::cout
    << "Hello world from process "
              << rank
              << " of "
              << size
              << std::endl;

    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output:



QUESTION 2

Write a programthat readsan integer value from the terminal and distributes the value to all of the MPI processes.Each process should print its rank and the value it received. Values should be read until a negativeinteger isgiven as input.

Theory / Explanation

This program demonstratesbroadcasting in MPI. Process 0 (the master process) reads an integer from the user. MPI_Bcast() thensends the same value from process 0 to every process in MPI_COMM_WORLD.Therefore, all processes receive the same value and print their rank along with that value.

The program keeps accepting values. If the value is negative, MPI_Bcast() distributes the negative value to all processes and every process exits the loop. A negative value is therefore used as the termination condition.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get process rank and total number of processes int
    rank, size;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size); int
    value;

    while (true)
    {
        // Process 0 reads the value
        if (rank == 0)
        {
            std::cout << "Enter an integer (negative number to stop): ";
            std::cin >> value;
        }

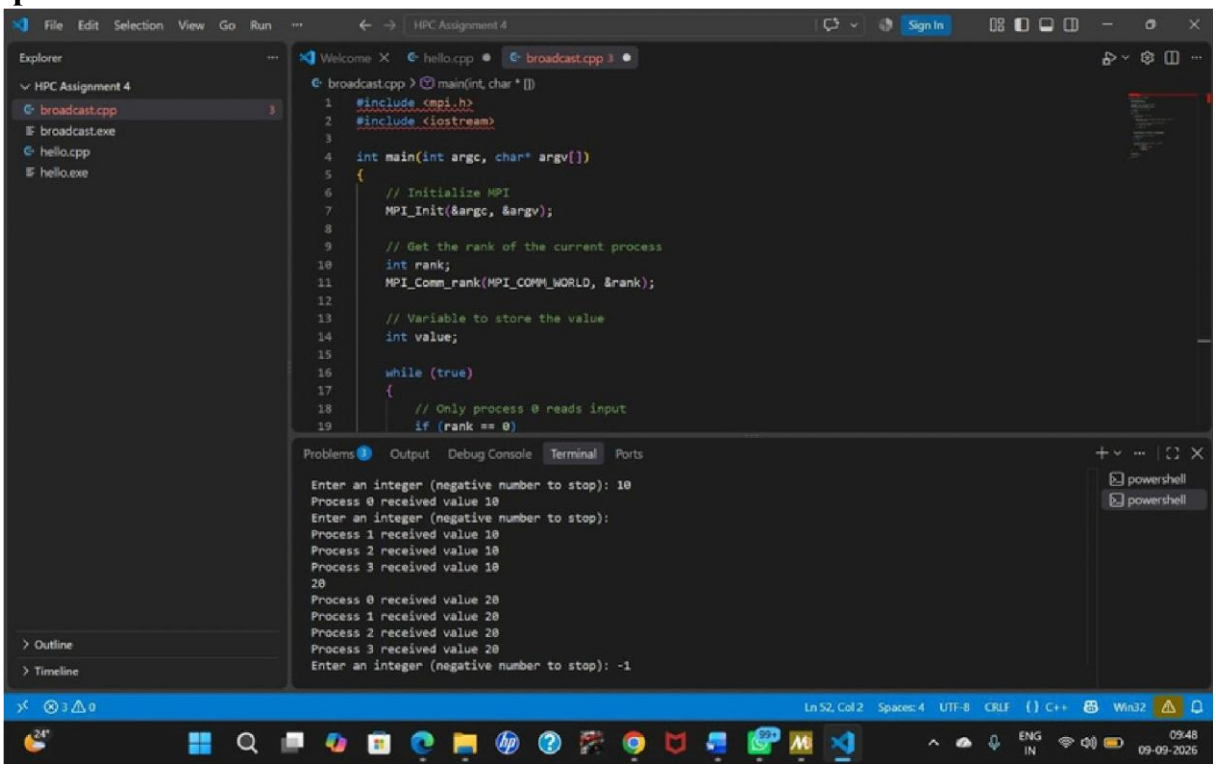
        // Distribute thevalue to all processes MPI_Bcast(&value,1,
        MPI_INT, 0, MPI_COMM_WORLD);

        // If the value is negative, stop all processes
        if (value < 0)
        {
        }break;
        //

           Each process prints its rank and the value received
        std::cout << "Process "<< rank
                  << " receivedvalue " << value << std::endl;
    }
    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output



QUESTION 3

Write a programthat takes data from process zero and sends it to all of the other processes by sending it in a ring. Process i should receive the data and send it to process i+1 until the last process is reached.

Theory / Explanation

A ring communicationpattern passes data from one process to the next process in sequence. Process 0 first reads the valueandsends it to process 1. Process 1 receives the value and sends it to process 2. This continues until thelast process is reached.

MPI_Send() is used tosend the value and MPI_Recv() is used to receive it. The destination of each processis its next rank.This demonstrates point-to-point communication and the ring topology in MPI.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and total number of processes int
          rank, size;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD,  &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size); int
    value;

    // Process 0 reads the data if
    (rank == 0)
    {
        std::cout << "Enter an integer: "; std::cin
        >> value;

        // Send data to process 1
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

        std::cout << "Process 0 sent value " << value
                  << " to process 1" << std::endl;
    }
    else
    {
        // Receive data from previous process
        MPI_Recv(&value, 1, MPI_INT, rank - 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        std::cout << "Process " << rank
                  << " received value " << value
                  << " from process " << rank - 1 << std::endl;

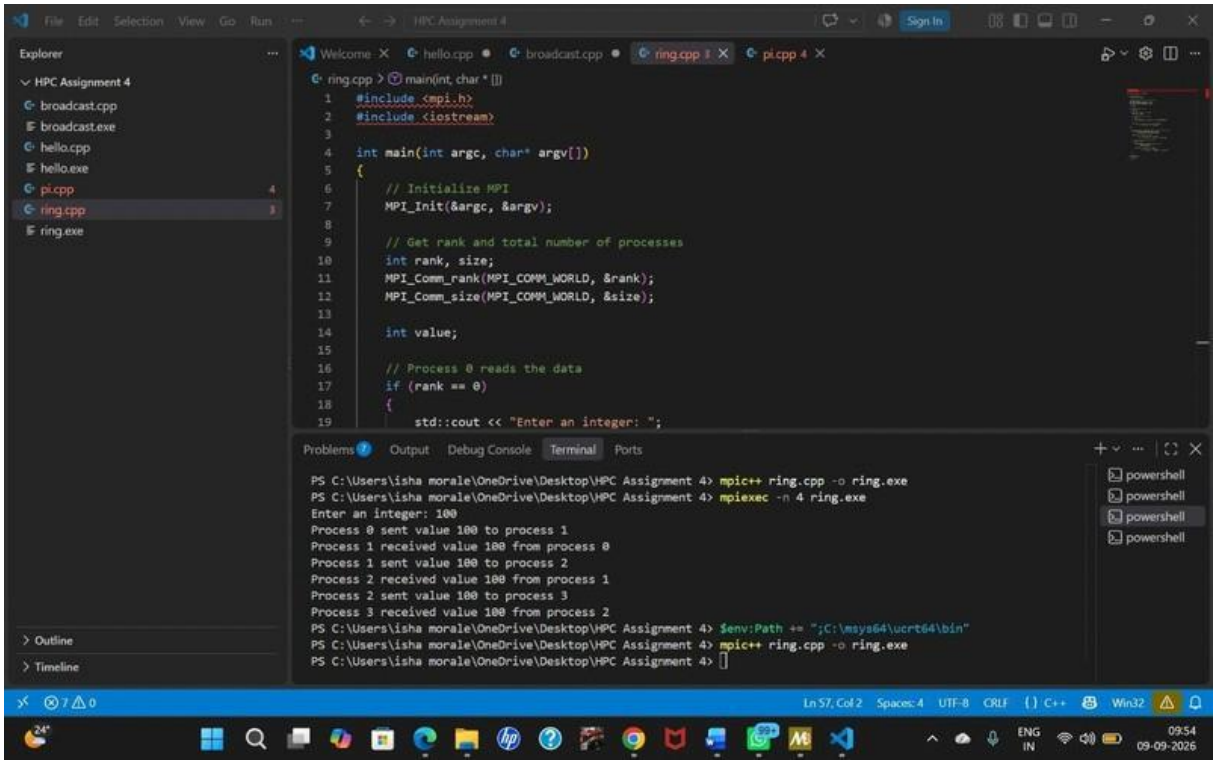
        // Send data to next process if it is not the last process
        if (rank < size - 1)
        {
            MPI_Send(&value, 1, MPI_INT, rank + 1, 0,
                     MPI_COMM_WORLD);

            std::cout << "Process " << rank
                      << " sent value " << value
                      << " to process " << rank + 1 << std::endl;
        }
    }

    // End MPI
    MPI_Finalize();

}   return 0;
```

Output



QUESTION 4

Write a simpleprogramtodetermine the value of pi. The method evaluates the integral of 4/(1+x*x) between 0and1.Theintegralisapproximated by a sum of n intervals. The master process (rank 0) asks the userforthenumberofintervals, broadcasts this number to all other processes, and each process addseverynthinterval.Finally, the partial sums are added using a reduction. Use MPI_Bcast and MPI_Reduce.

Theory / Explanation

The valueofpicanbeapproximated by evaluating the integral of $4/(1+x^2)$ from 0 to 1. The interval from 0 to1isdividedintonsmallintervals. The width of every interval is 1/n, and the contribution of an intervaliscalculatedusing$4/(1+x^2)$.

Process 0askstheuserforthenumber of intervals. MPI_Bcast() broadcasts this number to all processessothateveryprocessknows the same value of n. The processes divide the work by calculatingeverysize-thinterval,starting at its own rank. Each process obtains a partial sum. MPI_Reduce()withMPI_SUMadds all partial sums and places the final result at process 0.

For example,withfourprocesses,process 0 handles intervals 0, 4, 8, ...; process 1 handles 1, 5, 9, ...; process2handles2,6,10,...;andprocess 3 handles 3, 7, 11, ... This allows the numerical integration work to be performed in parallel.

Code

```
#include <mpi.h>
#include <iostream>
#include <iomanip>
```

```
int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get process rank and total number of processes
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Number of intervals int
    n;

    // Process 0 asks the user for the number of intervals if
    (rank == 0)
    {
        std::cout << "Enter the number of intervals: ";
        std::cin >> n;
    }

    // Broadcast n to all processes
    MPI_Bcast(&n, 1,            MPI_INT, 0, MPI_COMM_WORLD);

    // Calculate the double h = width of each interval
    1.0   /   double   local_sum static_cast<double>(n);
                               = 0.0;
    // Each process calculates every size-th interval for
    (int i = rank; i < n; i += size)
    { double x = h * (i + 0.5); local_sum
        += 4.0 / (1.0 + x * x);
    }

    // Multiply by interval width
    local_sum *= h;

    // Reduce all partial sums to process 0
    double total_sum = 0.0;
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE,
               MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    // Process 0 prints the final value of pi
    if (rank == 0)
    { std::cout << std::setprecision(15);
                                                    "
        std::cout << "Calculated value of PI =
                  << total_sum << std::endl;
    }

    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output

Conclusion

The four MPIprograms demonstrate basic parallel programming concepts: process identification, broadcastingdata to all processes, point-to-point communication in a ring, and parallel numerical computationusing reduction. MPI_Bcast was used for distributing common data, while MPI_Reduce was used tocombine partial results. The programs were compiled with mpic++ and executed using mpiexec withfour processes.

